6.046 Lecture #20 and #21

Today we switch to the new topic of TRACTABLE vs INTRACTABLE problems.

The complexity of all problems studied thus far were polynomial time. Namely, all problems had algorithms which ran in worst case time T(n) = O(n^k) for some fixed k>0 where n is as usual the size of the input. In fact, none of our running time were worse than O(n^3). We consider problems with such running time O(n^k), TRACTABLE, or efficiently solvable problems.

It may seem to you that when k=100 say, it hardly constitutes an efficient solution. That may be so, but certainly a problem whose running time cannot even be bounded by O(n^k) for ANY k is INTRACTABLE.

A natural question is whether all problems are TRACTABLE? The answer is No. A famous intractable problem is the HALTING problem. Given a program P and an input to it x, does P(x) ever halt? It turns out we cannot design an algorithm which on input P and x will answer this question. Not in polynomial time, not in exponential time, not in principal. Why? Goto 6.045.

In this course, we will be interested in problems which are solvable in principal but may take exponential time O(2^{n^k}) for some k>0 rather than polynomial time O(n^k).

We will restrict our attention to DECISION PROBLEMS. These are problems for which the output is always either YES or NO.

Formally, a decision problem D is a function from the set of all strings (all possible inputs) to the set {1,0}. We shall call an input x to D a yes-input if and only if D(x) = 1, and otherwise we shall call x a no-input.

Examples of Decision Problems.

PATH PROBLEM

INPUT: Graph G=(V,E), and vertices s,t in V.
QUESTION: Is there a path from s to t in G such that the length of the path is <= |V| (the number of vertices).
OUTPUT: YES if such a path exists, and NO otherwise.

Notation: PATH(G,s,t) = 1 if and only if there is a path from s to t in G where the length of the path < number of vertices in the graph.

Clearly, algroithms for shortest paths we have learned recently can be used to quickly in O(m+n) answer this question where m is the number of edges. Thus, this problem is tractable.

More generaly, we let
P = {Decision problems D for which there exist an polynomial time
algorithm A such that A(x) = YES if and only if D(x) =1}

CLIQUE PROBLEM

INPUT: Graph G=(V,E) and bound B.
QUESTION: Is there a subset C of the vertices V such that |C| >= B and for all u,v in C, (u,v) is in E. Such a set is called a clique of size B in G.
OUTPUT: Yes if such a large clique exists, and NO otherwise.

Notation: CLIQUE(G,B) =1 if and only if there is a clique of size B in G.

TRAVELING SALESMAN PROBLEM(TSP)

INPUT: Weighted graph G=(V,E), and bound B.
QUESTION: Does there exist a tour through the graph that visits all vertices exactly once, starts and ends at the same vertex, and is of cost <= B.
OUTPUT: YES if such a cheap tour exists, and NO otherwise.

Notation: TSP(G,B) = 1 if and only if there is a tour of cost <= B.

3SAT

INPUT: Formula of n Boolean variables f(x1,...xn) which is in the form of a conjuction of clauses each being a disjunction of 3 of the variables each appearing either in a negate or non-negated form.
e.g. f(x1,x2,x3) = (x1 OR not x2 OR x3) (x1 OR x2 OR not x3)
QUESTION: Is there a Boolean assignment to x1,...,xn that would make f(x1,...,xn) = TRUE (this is called a satisfying assignment).
OUTPUT: If a satisifying assignment exists YES, else NO.
Notation: 3SAT(f) = 1 if and only if there is a satisfying assignment to f.

Whereas PATH was easy to solve, we know of no efficient polynomial-time solution for CLIQUE, TSP, and 3SAT. In fact (looking ahead) CLIQUE, TSP, and 3SAT are all NP-complete problems.

NP-complete problems are a collection of problems from as varied areas as graph theory, logic, number theory, algebra, and combinatorices which all share the following properties:

- -The best known algorithms to solve any known NP-complete problem take O(2^{n^k}) time for some fixed k>0.

- -If a polynomial time algorithm will be found for any NP-complete problem, then all NP-complete problems could be solved in polynomial time.

The prevailing belief after 30 years of work, is that NP-complete problems do not have polynomial time algorithms, and thus are not in P.

So, why do we study them in this course which is dedicated to the design of algorithms?

Good question...A few answers...

1. It is good to be able to recognize an NP-complete problem when you encounter one. This way you wont waste time trying to design an polynomial time algorithm for it.

2. We shall learn about desiging approximation algorithm for NP-complete problems which will run in polynomial time. An approximation algorithm will give an answer which is an approximation of the true answer. This does not make much sense for decision problems but will make sense when we consider the search problem versions of the decision problems (see below).

3. Change your problem formulation to put it in P rather NP-complete. For example, restrict the set of inputs for which the problem should be solved. For example, for TSP, a changed fomulation restricts the input graphs to planar graphs. Unfortunately, planar-TSP is still NP-complete but we shall see nice approximation algorithms for it which run in polynomial time.

4. Fame and ``Greed''. This question of whether NP-complete problems are in P or not is the most famous problem in computer science. There is nice prize to whomever resolves it. See www.claymath.org/prizeproblem (also contains exposition of this problem).

5. This is the only theory course you are REQUIRED to take, although I encourage you to take more. You must know about the issue of classifying computational problems by their complexity as an educated computer scientist.

Note that so far we have not formally defined NP-complete problems.

To do this, we shall first define NP and the notion of `reductions'.

Intuitively, we say that a decision problem D is in NP if (regardless whether it is easy or hard to solve) it has the following property:

> For every yes-input x to D, there is a polynomial size piece of evidence_x which can be checked in polynomial time that indeed x is a yes-input. This `evidence' (sometimes called a `certificate' or `witness' of `proof') may be very hard to come up with, but is easy to check.

For example,

CLIQUE is in NP. Why? Let G=(V,E). When CLIQUE(G,B) = 1, the subset S of V which is a clique of size greater or equal to B is itself the `evidence' that CLIQUE(G,B)= 1. The `evidence' S is of size <= |V| and can be checked in O(|V|^2) time.

TSP is in NP. Why? Let G=(V,E). When TSP(G,B) =1, the ordertour of vertices by which the tour of cost B proceeds, is the `evidence' that TSP(G,B)=1. The tour is of size |V| and the cost of the tour can be computed in O(|V|) time.

3SAT is in NP. Why? When 3SAT(f) = 1, the assignment to the variables

x1,...,xn which makes f true is the `evidence' that 3SAT(f) =1. The size of the assignment is n and evaluating the formula is linear in its size.

Formally,

NP = { Decision problems D such that there exists A and polynomials p,q such that for |x|=n, D(x) = 1 if and only if there exists y such that |y|<p(n) s.t. A(x,y) = YES and the running time of A(x,y) is bounded by q(n)} (Note: y is the `evidence' that D(x) =1.)

THEOREM: P subset of NP

This is obvious. If you can solve a problem efficiently, there exists short evidence as to what the solution is (both for YES and NO inpus !). Simply, use the execution trace of your solving algorithm as the evidence that the answer is YES or NO depending on what the case may be.

REDUCTIONS

Informally, we say that problem A is polynomial time reducible to B if, there exist a function R which takes an input x to problem A and transforms it to an input R(x) to problem B such that x is a yes-input of A if and only if R(x) is a yes-input of B; and R is a polynomial time computable.

Formally, we say that problem A <p B if: there exist a polynomial time computable function R:{0,1}^* -> {0,1}^* such that A(x) = 1 if and only if B(R(x))=1.

CLAIM: if A <p B and B is in P, then A is in P.
PROOF: Say R is the polynomial time reduction of A to B and it takes q(m) time. Say the algorithm for B runs in time p(n).

Here is a polynomial time procedure for A.

```
        On input x.
        Transform it to R(x).            /* cost is q(|x|)   */
        Run the algorithm for B on R(x)  /* cost is p(q(|x|)) */
        Output B(R(x)).
```

The total runtime is q(|x|) + p(q(|x|)) which is a polynomial in |x| as both p and q are polynomial functions.

CLAIM (transitivity): if A <p B and B<p C then A <p C. Proof: Left for you.

NP-Completeness

We are finally ready to define NP-completeness. Those problems in NP which are "as hard as" any other problem in NP. More accurately, any input to any NP problem can be trasformed to (or expressed as) an input of an NP complete problem so that the YES/No answers are preserved.

We say that a problem A is NP-complete if
1. A is in NP
2. for all decision problems C in NP, C<pA.
(this requirement alone is called NP-hardness)

THEOREM: If A is NP-complete and A is in P, then NP=P

The first problem proved to be NP-complete was by Steve Cook in 1974 and was the SAT problem (like 3SAT above without te restriction of 3 variables per clause).

A high level description of the proof is as follows. For any problem C in NP there exists a polynomial time verification algorithm A(.,.) (by definition of C in NP). To show a reduction from C<p SAT, Cook shows how to convert every input x to C into a logical formula f_x so that there exists a y that makes A(x,y) = YES if an only if there exists a truth assignment z that makes f(z) = TRUE.

The proof is beautiful, and involves using a fixed model of computation for A such as Turing Machines and showing that its computation can be expressed as a logical formula in 3CNF (conjunctive normal form). It is done in detail in 6.045. Here we shall just assume 3SAT is NP-complete. And use this fact to prove NP-completeness for many other problems (in a much simpler way).

What is a strategy to prove NP-completeness for a new decision problem B.

1. Prove that B is in NP.
2. Prove, for an already known NP-complete problem such as 3SAT, that 3SAT <p B.
By transitivity of reductions, this means that for all C in NP, C <p B.

Thus, B is NP-complete.

We shall see several proofs of this form.

Theorem: Clique is NP-complete

Proof: We already saw above that CLIQUE is in NP.

Lets see how to prove 3SAT <p Clique.

We need to transform input formula f in 3CNP form f = C1 and C2 and.... Cm on n variables x1....xn, into a pair (G,B) such that 3SAT(f) =1 iff CLIQUE (G,B)=1.

The graph G is defined as follows.

VERTICES: for each clause Ci, insert 7 new vertices (we will refer to those as the clause Ci vertices) corresponding to the 7 assignments to the variables in clause Ci that will make this clause be TRUE (i.e satisfied). Note, that this are partial assignment as they give value only to the 3 variables which appear in the clause and not to the rest of the variables. Thus, there is a total of 7m vertices.

EDGES: For every pair of vertices u,v: if both u and v are clause Ci vertices, do not put an edge between them. If u is a clause Ci vertex and v is a cluase Cj vertex, put an edge between them only if the partial assignments they contain are consistent with each other. Namely if variable x=True in vertex u and x=False in vertex v do not put an edge between u and v.

The value B = m = the number of clauses in f.

Facts: (1) The size of the graph is O(m^2)
(2) If 3SAT(f) = 1 then CLIQUE(G,m) =1
(3) If Clique(G,m) = 1 then 3SAT(f) = 1

Proof:
(1) by construction.

(2) If 3SAT(f)=1 it means that there exists an assignment x1...xn to the variables that makes f true, i.e it makes each of the clauses C1....Cm true. Consider the following set of vertices S = {for each clause Ci insert the clause Ci vertex whose partial assignment is consistent with the assignment x1...xn}. Note that |S|=m as there is one vertex per clause in S. And its a clique because there is an edge between any pair of vertices in S as they all are consistent with the unique x1....xn and thus they are consistent between themselves.

(3) If CLIQUE(G,m)=1 it must be that the clique contains exactly 1 vertex from the 7 vertices of clause i for each i=1...m (this is so as there are no edges between two different clause i vertices). Now, make the assignement to the variables of clause i, be the one specified by the vertex of clause i which is in the clique. It will satisfy clause Ci by definition. Moreover, the partial assignments you have thus made are consistent across all clauses with each other as otherwise they would not have an edge between them and would not form a clique to begin with.

QED

We can now show that new problems are NP-complete by showing either (1) 3SAT <p New-Problem or (2) CLIQUE <p New-Problem.

It is easy to see this way that CLIQUE <p IS <p VC where

Independent Set (IS)

INPUT: Graph G and bound B.
QUESTION: Does there exist a subset of vertices of size larger or equal to B such that any 2 verties do not have an edge between them?

Vertex Cover (VC)

INPUT: Graph G and bound B
QUESTION: Does there exist a subsect of vertices of size smaller or equal to B such that all edges in the graph have at least one (maybe two) end point which is in the subset.

To show CLIQUE <p IS simply reduce (G,B) to (G complement, B) where G complement has the same vertex set as G but the complement set of edges.

To show IS <p VC simply reduce (G,B) to (G, |V|-B) . Prove this is a working polynomial time reduction!